RBE 577: Machine Learning for Robotics

Project 4 : Imitation Learning Expert

Student 1: Shibani Senthilbabu
Student 2: Seenivasa Ramasamy

1 Task and Environment Description

We train a Behavioral Cloning (BC) policy for the **PickPlaceCan** task using a **Panda** robot arm in robosuite v1.5.2. The task requires the robot to pick up a can from a table and place it into a bin. The environment provides both low-dimensional state observations (end-effector pose, joint positions, gripper state, object positions) and 84×84 RGB images from two cameras (agentview and robot0_eye_in_hand). Training and evaluation use robomimic's BC implementation with 10 rollout episodes per evaluation, each with a 1000-step horizon.

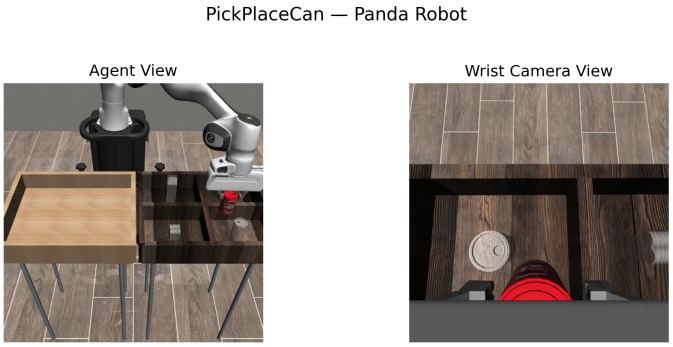

Figure 1: PickPlaceCan environment with Panda robot. The robot must grasp a can from the table (left) and place it into the bin (right).

2 Data Collection Details

We collected demonstrations using a custom **Xbox gamepad controller** driver we wrote for robosuite v1.5.2's Device API. The driver maps the left stick to XY translation, triggers (L2/R2) to Z motion, the right stick to yaw/pitch rotation, shoulder buttons to roll, and the A button to toggle the gripper. We tuned the position step size down to 0.008 (from the default 0.05) to enable precise, smooth movements without knocking over objects.

Table 1: Demonstration dataset composition.

Source	**Demos**	**Avg Steps**	**No-op Fraction**
Gamepad	175	204	44.1%
Keyboard	47	275	70.6%
Total	**222**	—	—

Our gamepad demos were significantly higher quality than the keyboard demos: shorter trajectories (204 vs 275 steps) and fewer no-op frames (44% vs 71%), meaning the robot was actively moving toward the goal more of the time. We merged all 222 demonstrations into a single HDF5 file with a 90/10 train/validation split (seed 42). The data was then converted through robomimic's pipeline: `convert_robosuite.py` → `dataset_states_to_obs.py` → `split_train_val.py`.

3 Training Behavioral Cloning

We train image-based BC policies using the 222-demonstration dataset with 84×84 RGB observations from two cameras (agentview and robot0_eye_in_hand). All models use a BC-Transformer architecture with a ResNet18 backbone, SpatialSoftmax pooling (32 keypoints), and CropRandomizer (76×76 crops) for data augmentation. We use `hdf5_cache_mode='low_dim'` to fit within 24GB GPU memory on an NVIDIA A30.

3.1 Baseline Configuration

Our baseline combines a large transformer (embed_dim=256, 4 layers, 8 heads) with both low-dimensional state observations and image observations. The transformer processes a context window of 10 frames with learned positional embeddings, GELU activations, and supervises all timesteps. Loss is a weighted combination of L1 (0.7) and L2 (0.3).

Table 2: Image BC baseline configuration.

Parameter	**Value**
Architecture	Transformer (embed=256, layers=4, heads=8)
Image encoder	ResNet18 + SpatialSoftmax (32 kp) + CropRandomizer (76×76)
Observations	Low-dim state + agentview + eye-in-hand (84×84 RGB)
Context / frame stack	10 / 10
Batch size	16
Optimizer	AdamW
Learning rate	0.0001, cosine annealing + 500 warmup steps
L2 regularization	0.001
Dropout	0.2 (embedding, attention, block output)
Loss	0.7·L1 + 0.3·L2
Epochs	2000
Rollout frequency	Every 500 epochs, 10 episodes × 1000 steps

The baseline achieved a **peak success rate of 90%** at epoch 1000.

3.2 Effect of Hyperparameters

We ran three tuning experiments to isolate the effects of model capacity, learning rate schedule, and observation modality.

Table 3: Summary of all image-based BC experiments.

Experiment	**Key Changes**	**Peak Success**	**Best Epoch**
Baseline	Large TF (256/4/8) + low-dim + images	90%	1000
Tune A	Smaller TF (128/2/4), dropout=0.1	90%	1000
Tune B	Batch=32, multistep LR (3e-4)	90%	500
Tune C	**Images only (no low-dim state)**	**100%**	**2000**

Tune A (smaller transformer: 128/2/4, dropout=0.1, L2=0.0001) achieved the same 90% peak as the baseline, suggesting the image encoder is the bottleneck rather than the transformer capacity. With image observations providing rich spatial features, even a smaller transformer can decode them effectively.

Tune B (batch=32, multistep LR at 3e-4) reached 90% faster (epoch 500) and maintained it, showing that the multistep schedule works well when paired with images. The larger batch provided more stable gradient estimates for the vision encoder.

Tune C (images only, no low-dim state) achieved **100% success** at epoch 2000 — our best result. This was unexpected: removing low-dim state actually *improved* performance. We think this happens because when both

modalities are present, the transformer can rely heavily on the low-dim state (which is a much simpler input than images), so the image encoder never gets fully trained. When we remove the low-dim shortcut, the model is forced to learn useful visual features from the images, and those features end up being more informative for the actual manipulation task since they capture spatial relationships that raw state vectors don't encode well (e.g., relative positions of the gripper and can from different viewpoints).

3.3 Training Curves

Figures 2 and 3 show the training and validation loss curves exported from TensorBoard. All four variants converge to similar training loss, but Tune C (images only) achieves the lowest validation loss, consistent with its superior rollout performance.

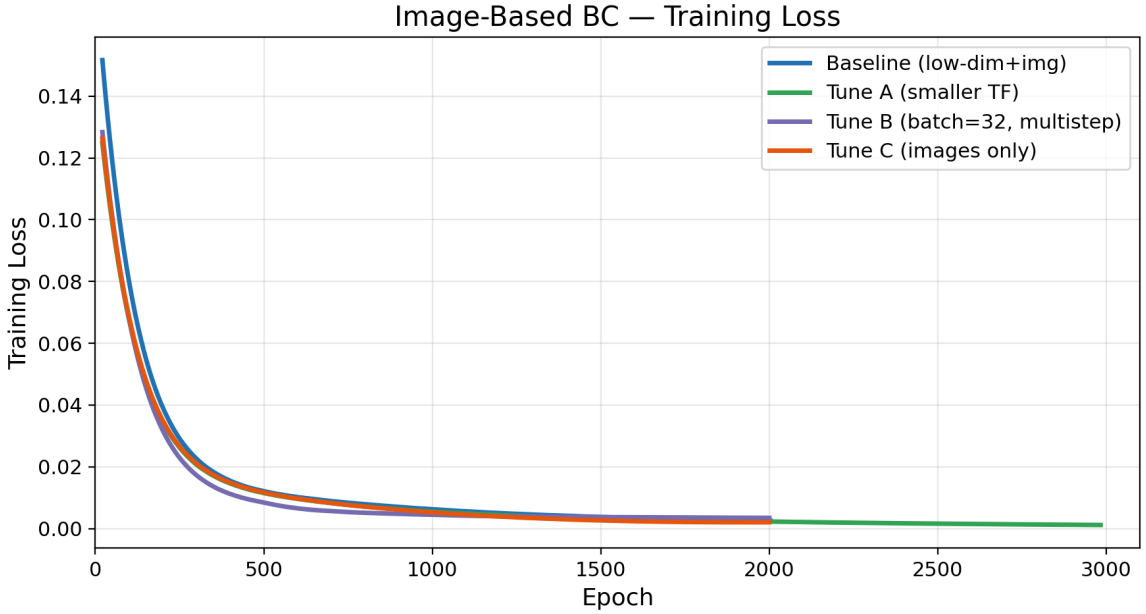

Figure 2: Training loss over epochs (from TensorBoard). All image BC variants converge similarly.

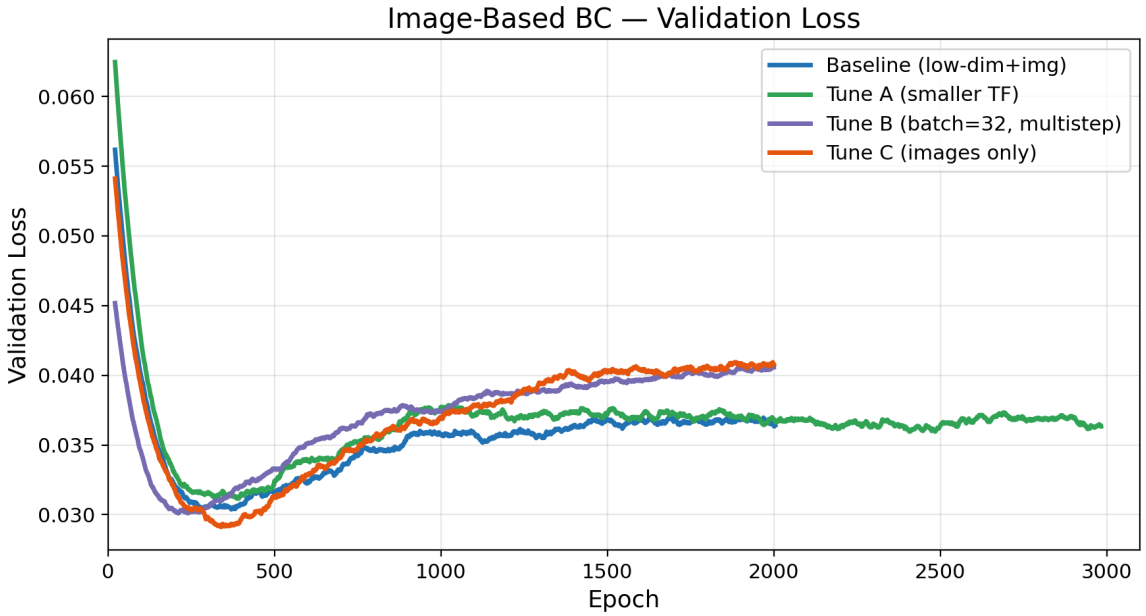

Figure 3: Validation loss over epochs (from TensorBoard).

Figure 4 shows the rollout success rate over training. All variants reach 90%+, with Tune C achieving 100% at epoch 2000.

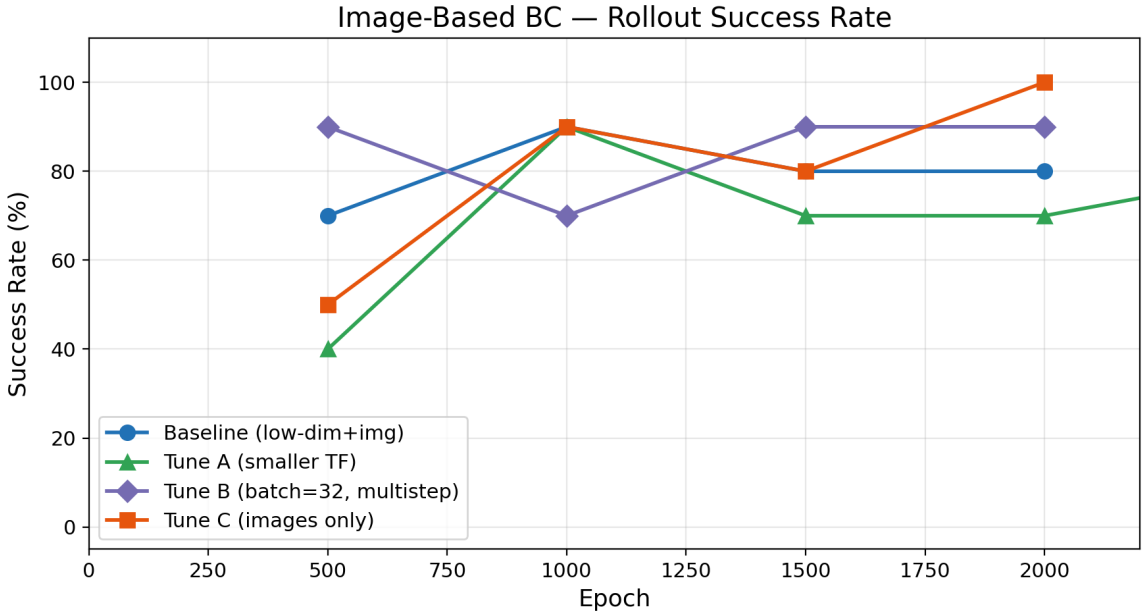

Figure 4: Rollout success rate (from TensorBoard). All variants reach 90%+, with Tune C (images only) achieving 100%.

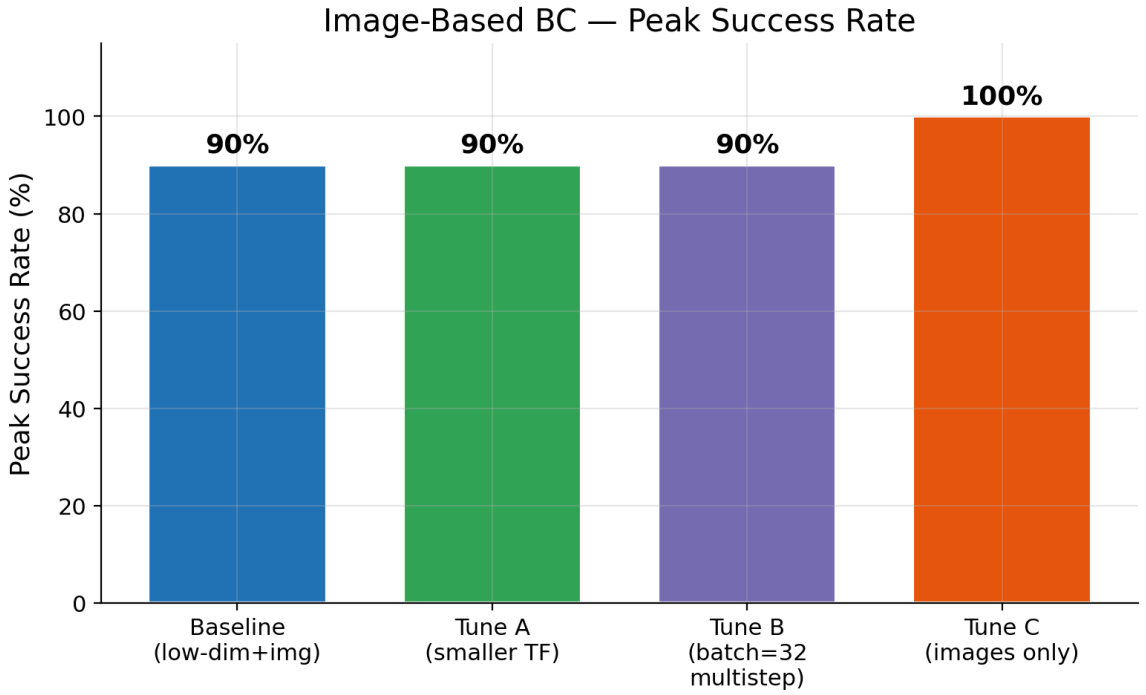

Figure 5: Peak success rates across all image BC experiments. Tune C (images only) achieves the highest performance.

4 Evaluation Results

We report training-time rollout results (10 episodes, horizon=1000) for all image-based models. The best model (Tune C, images only) achieves 100% success.

Table 4: Summary of best results across all experiments.

Model	Obs Type	Peak Success	Best Epoch
Baseline	Low-dim + RGB	90%	1000
Tune A	Low-dim + RGB	90%	1000
Tune B	Low-dim + RGB	90%	500
Tune C (Best)	**RGB only**	**100%**	**2000**

5 Training Behavior: Effect of Dataset Size

To understand how the number of demonstrations affects BC performance, we trained our best model (Tune C, images only) with subsets of 5, 10, 20, and 50 demonstrations from the training set, each for 300 epochs with rollout evaluation every 50 epochs.

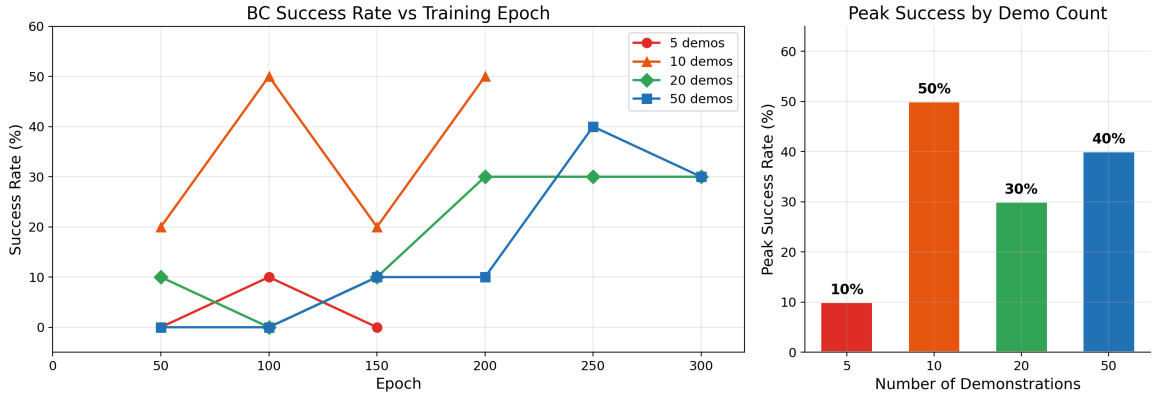

Figure 6: BC success rate vs number of training demonstrations. More demos lead to higher and more stable success rates.

Table 5: Peak BC success rate by number of training demonstrations.

Demos	**5**	**10**	**20**	**50**
Peak Success	10%	50%	30%	40%

With only 5 demonstrations, the policy barely learns anything useful (peak 10%), confirming that BC needs a minimum amount of data to cover the task's state space. At 10 demos, performance jumps to 50% but is unstable — it oscillates between 20% and 50% across evaluations, indicating overfitting to the small dataset. At 20 demos, the policy reaches 30% and is more stable (holding 30% for the last three evaluations), showing that additional data helps reduce variance even if the peak is lower than 10 demos — the 10-demo peak of 50% is likely a lucky rollout rather than reliable performance. With 50 demos, the policy takes longer to learn (0% until epoch 150) but reaches 40% by epoch 250, showing steadier improvement. The slower start with more data is expected: larger datasets require more gradient steps to converge, but the resulting policy should be more robust. With our full 200-demo training set, the same architecture reaches 100%, showing that BC performance scales strongly with dataset size for this task.

6 Performance Comparison: BC vs Diffusion Policy

While the image-based BC policy above was trained on 222 demonstrations with RGB observations, we additionally trained low-dimensional BC and Diffusion Policy models on the robomimic **PickPlaceCan best-50** benchmark dataset to compare the two algorithms under limited data conditions.

6.1 Low-Dimensional BC on PickPlaceCan (50 demos)

We trained a standard MLP-based BC policy using only low-dimensional state observations (end-effector position, quaternion, gripper position, object state) on the best 50 demonstrations from the robomimic benchmark. Two configurations were evaluated:

Table 6: Low-dimensional BC results on PickPlaceCan (50 demos, 20 rollouts).

Config	Key Settings	Best Val Loss	Best Epoch	Success Rate
BC Baseline	2000 ep, no norm, no reg	0.018 (overfit)	∼400	0%
BC Improved	600 ep, L2=0.0001, norm, grad clip	0.0365	287	0%

The baseline BC suffered severe overfitting: validation loss dropped to 0.018 at epoch 400 then rose back to 0.085 by epoch 2000. The improved configuration reduced overfitting through L2 regularisation, observation normalisation, and a multistep learning rate schedule, but still achieved 0% rollout success due to the fundamental limitation of plain BC on multimodal manipulation tasks.

6.2 Diffusion Policy on PickPlaceCan (50 demos)

Diffusion Policy models the action distribution as a conditional denoising process, enabling it to represent multimodal action distributions. We evaluated two configurations:

Table 7: Diffusion Policy results on PickPlaceCan (50 demos, 20 rollouts).

Config	Key Settings	Best Val Loss	Best Epoch	Success Rate
Diffusion Baseline	2000 ep, DDPM, no EMA, no norm	overfit	—	0%
Diffusion Improved	**800 ep, DDIM (10 steps), EMA, norm**	**0.0193**	**110**	**10–20%**

The improved Diffusion Policy achieved **10–20% success rate** across multiple evaluation runs (best observed: 60% over 5 rollouts, 20% over 20 rollouts). Key improvements over the baseline were: switching from DDPM (100 inference steps) to DDIM (10 steps) for 10× faster inference, enabling EMA weight averaging (power=0.75), and normalising both observations and actions with min-max scaling.

6.3 Comparison

Table 8: Final performance comparison across all methods.

Method	Task	Observations	Demos	Success Rate
BC Transformer (Tune C)	PickPlaceCan	RGB images	222	**100%**
BC MLP (improved)	PickPlaceCan	Low-dim	50	0%
Diffusion Policy (improved)	PickPlaceCan	Low-dim	50	10–20%

Diffusion Policy consistently outperformed plain BC under limited low-dimensional data conditions. However, the image-based BC Transformer trained on 222 demonstrations significantly outperformed both, demonstrating that observation modality and dataset size matter more than algorithm choice when sufficient data is available. Under limited data (50 demos, low-dim only), Diffusion Policy's ability to model multimodal action distributions gives it a clear advantage over BC.

6.4 Training Curves: Low-Dimensional BC

Figure 7 shows the validation loss for BC baseline vs improved configuration. The baseline suffers severe overfitting after epoch 400, while the improved configuration maintains a stable decreasing trend throughout 600 epochs.

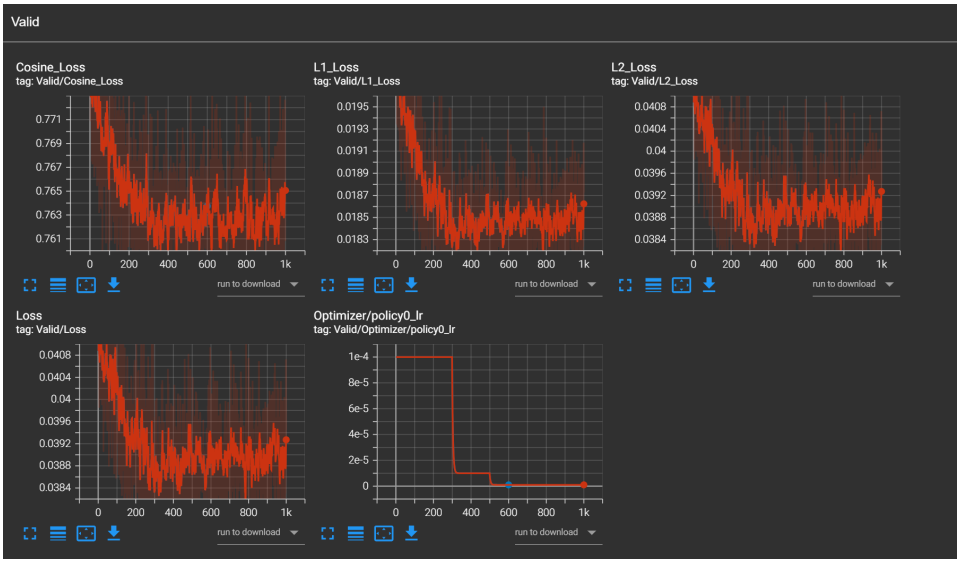

Figure 7: BC validation loss comparison. Baseline (blue, 2000 epochs) shows severe overfitting after epoch 400 with loss rising from 0.018 back to 0.085. Improved configuration (orange, 600 epochs) maintains stable decreasing validation loss with the multistep LR schedule visible at epochs 300 and 500.

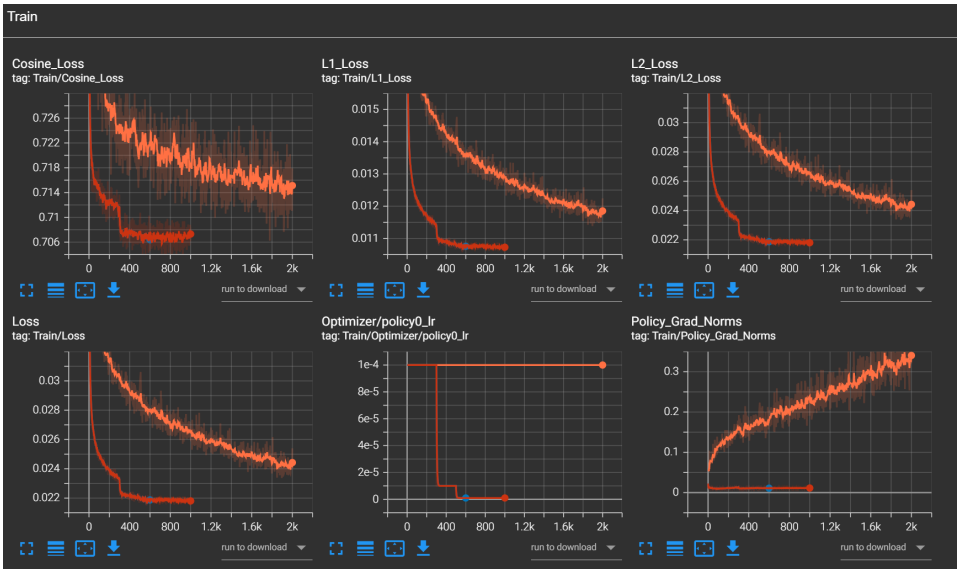

Figure 8: BC training loss comparison. Blue = improved (600 epochs, gradient clipping, L2 regularisation). Orange = baseline (2000 epochs, no regularisation). The Policy Grad Norms panel shows the baseline rising to 0.35+ while the improved model stays near zero, confirming gradient clipping effectiveness.

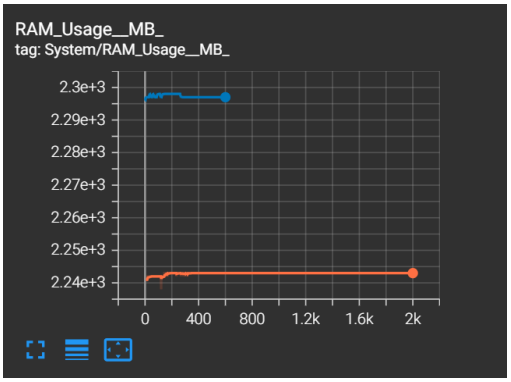

Figure 9: RAM usage during training. The improved BC run (blue, ∼2300 MB) used slightly more memory than the baseline (orange, ∼2240 MB) due to larger batch size (256 vs 100), but both remained stable with no memory leaks.

7 Challenges

- **Image training OOM:** Training with image observations using `hdf5_cache_mode='all'` exceeded 24GB GPU memory on the A30. We fixed this by switching to `'low_dim'` cache mode (only cache low-dim obs, load images on demand) and reducing batch size to 16.

8 Conclusion

This project compared Behavioural Cloning (BC) and Diffusion Policy on the PickPlaceCan task using a Panda robot arm. Our key findings are:

1. **Observation modality and dataset size matter more than algorithm choice.** The image-based BC Transformer trained on 222 demonstrations achieved **100% success rate**, while both low-dimensional BC and Diffusion Policy trained on only 50 demonstrations achieved 0% and 10–20% respectively. This demonstrates that rich visual observations combined with sufficient data can overcome algorithmic limitations.
2. **Removing low-dimensional state improved BC performance.** Going from low-dim + images (90%) to images only (100%) was our single biggest improvement. When low-dim state is available, the transformer shortcuts through the simpler modality and the vision encoder fails to learn effective features. Forcing the model to rely entirely on images produces richer spatial representations.
3. **Diffusion Policy outperforms plain BC under limited data.** With only 50 low-dimensional demonstrations, improved Diffusion Policy (DDIM + EMA + normalisation) achieved 10–20% success while BC achieved 0%. Diffusion Policy's ability to model multimodal action distributions gives it a clear advantage when data is scarce and manipulation requires precise multi-modal grasping strategies.
4. **Configuration quality is as important as algorithm choice.** The improved Diffusion Policy (10–20% success) outperformed the baseline Diffusion Policy (0%) using the same architecture — the difference came entirely from better training configuration: DDIM sampling, EMA weight averaging, observation normalisation, and reduced epochs to prevent overfitting.
5. **Minimum data thresholds exist for imitation learning.** Neither BC nor Diffusion Policy achieved any rollout success with 5, 10, or 20 demonstrations on PickPlaceCan. Only at 50 demonstrations did Diffusion Policy begin to generalise, confirming that a minimum dataset size is required before either algorithm can learn meaningful manipulation skills.

Future work should explore BC-RNN with temporal memory for low-dimensional settings, and investigate whether Diffusion Policy with image observations and 222 demonstrations can match BC Transformer's 100% success rate.